SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4304

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I P.Yaswanth Choudhary(192411136)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: P.Yaswanth Choudhary

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS:

1. HTTP Response Analysis

1) Identify any incorrect or invalid fields in the header.

The given HTTP response header is:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type:
text/html Content-
Length: -150
Connection: keep-alive
The following errors are present:

- Content-Type: The value is incorrectly formatted. It should be text/html on a single header line.
- Content-Length: The value -150 is invalid because the content length cannot be negative.
- Date: The weekday and date do not match. April 1, 2026, is a Wednesday, not a Monday.

2) Analyze the issue with Content-Length and explain its impact.

The Content-Length header specifies the size of the response body in bytes. A negative value such as -150 is invalid.
Its impact includes:

- The server response may be rejected or handled incorrectly by clients.
- The browser may not correctly determine the response body length.
- Communication between the client and server may fail or produce unexpected results.

The correct value should be the actual number of bytes in the response body.

3) Determine whether the status code matches the response correctness.

The status code 200 OK indicates that the request was successfully processed and the server is returning a successful response.
However, this status code does not guarantee that every response header is valid. The given response contains errors in its headers, so the response needs correction even though the status code indicates success.

4) Suggest corrections for the identified errors.

A corrected example is:
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
Here, 150 is an example only. The actual Content-Length must match the response body's byte size.

2. DOM Structure Analysis

The given DOM structure is:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

1) Identify missing or improperly closed elements.

The element is missing its closing tag before the tag.
The paragraph should be properly closed inside the element.

2) Analyze how the browser will interpret this structure.

Browsers use the HTML parsing rules to handle certain missing closing tags automatically. In this example, the browser will generally infer the end of the paragraph when it encounters the closing tag.
As a result, the browser may construct a usable DOM tree despite the incorrect markup. However, relying on automatic correction is not a good coding practice.

3) Determine the impact on rendering and layout.

- The page may still display the text HelloWorld correctly.
- Incorrect nesting can cause unexpected layout or styling behavior.
- CSS rules applied to the paragraph may not work as intended if the markup becomes more complex.
- Poorly structured HTML can make maintenance and debugging difficult.

4) Suggest corrections to make the DOM valid.

Corrected HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>HelloWorld</p>
  </div>
</body>
</html>
```

This version correctly closes all elements and maintains proper nesting.

HelloWorld

3. GET vs POST in Login System

1) Interpret the differences in data transmission between GET and POST.

GET and POST are HTTP methods used to send data between a browser and a web server.

GET Method	POST Method
Data is sent through the URL as query parameters.	Data is sent in the request body.
Data is visible in the URL.	Data is not normally visible in the URL.
Suitable for retrieving information.	Suitable for submitting data, such as login credentials.
Data may be stored in browser history and server logs.	Request body data is not normally stored in browser history as part of the URL.
Less suitable for sensitive information.	More suitable for submitting sensitive information when combined with HTTPS.

2) Analyze which method is more suitable for login and why.

The POST method is more suitable for login systems because usernames and passwords are sent in the request body rather than appearing in the URL.
This helps prevent credentials from being exposed through browser history, copied URLs, and URL-based logs.
However, POST alone does not provide encryption.

3) Identify security risks in using GET for authentication.

Using GET for authentication has several security risks:

- Passwords may appear in the browser's address bar.
- Credentials may be saved in browser history.
- URLs containing passwords may be recorded in server logs.
- Shared or copied URLs may expose login details.
- Sensitive information may be disclosed through URL-sharing mechanisms.

4) Suggest best practices for secure login implementation.

The following practices should be followed:

1. Use the POST method to submit login credentials.
2. Use HTTPS to encrypt data transmitted between the browser and server.
3. Store passwords securely using strong password-hashing algorithms.
4. Validate login credentials on the server side.
5. Use secure session management and cookies.
6. Implement rate limiting to reduce brute-force login attempts.
7. Never store passwords in URLs or expose them in application logs.

Conclusion: POST combined with HTTPS and proper authentication practices provides a more secure login system than GET.

4. HTML Code Inspection

The given HTML code is:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1) Identify missing closing tags.

The following closing tags are missing:

- </h1> to close the heading.
- </p> to close the paragraph.

The <img> element is a void element in HTML, so it does not require a closing tag.

2) Analyze issues with the <img> tag.

The image tag is written as:

<img src="image.jpg">

The following issues should be considered:

- The alt attribute is missing. It should describe the image for accessibility and provide alternative text if the image cannot load.
- The file image.jpg must exist at the specified location; otherwise, the image will not display.
- The image's dimensions can be specified when necessary to help reduce layout shifts.

3) Determine how browsers will handle these errors.

Modern browsers are designed to handle many HTML errors automatically.

- The browser will generally infer the end of the heading when it encounters the paragraph.
- It will generally infer the end of the paragraph before the image or when the body ends.
- The page may still render, but missing accessibility information and incorrect markup can cause problems.
- If the image file is unavailable, the browser will not display the intended image.

Even though browsers can recover from these errors, writing valid HTML is recommended.

4) Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>
<head>
   <title>Sample</title>
</head>
<body>
   <h1>Welcome</h1>
   <p>This is a paragraph.</p>
   <img src="image.jpg" alt="Sample image">
</body>
</html>
```

This corrected version has properly closed elements and an appropriate alt attribute.

Welcome

This is a paragraph.

Sample image

5. Browser-Server Communication Flow

1) Interpret each step in the communication flow.

The browser-server communication process consists of the following steps.

Step 1: User enters a URL in the browser

The user enters a website address, such as www.example.com, into the browser's address bar.

Step 2: DNS lookup occurs

The Domain Name System (DNS) translates the domain name into an IP address that identifies the server.

Step 3: HTTP request is sent

The browser establishes the necessary network connection and sends an HTTP request to the server. For HTTPS, a secure TLS connection is also established.

Step 4: Server processes the request

The server receives the request, executes the required application logic, retrieves the necessary data or resources.

Step 5: Response is returned

The server sends an HTTP response containing a status code, headers, and, when applicable, a response body such as HTML content.

Step 6: Browser renders the page

The browser processes the HTML, CSS, and JavaScript, requests additional resources when needed, and displays the webpage to the user.

2) Identify where delays can occur.

Delays can occur at several stages:

- DNS lookup: The browser takes time to resolve the domain name.
- Connection establishment: Creating a TCP connection and establishing TLS for HTTPS can introduce delay.
- Request transmission: Network congestion or slow internet connections can delay requests.
- Server processing: Complex application logic or slow database queries can increase response time.
- Response transmission: Large files and limited network bandwidth can delay downloads.
- Browser rendering: Large HTML documents, complex CSS, and heavy JavaScript can slow page display.

3) Analyze the role of DNS in the process.

DNS stands for Domain Name System. Its main role is to translate human-readable domain names into IP addresses. For example, when a user enters www.example.com, DNS helps the browser identify the IP address associated with that domain.

DNS is important because:

- It allows users to access websites using domain names instead of numerical IP addresses.
- It helps the browser locate the correct server.
- DNS caching can reduce repeated lookup time.
- Incorrect DNS configuration or slow DNS responses can prevent or delay access to websites.

4) Suggest optimizations to improve performance.

The following techniques can improve browser-server communication:

1. Use DNS caching: Store DNS lookup results temporarily to reduce repeated lookups.
2. Optimize server performance: Improve application code and database queries.
3. Use HTTP caching: Cache static resources such as images, CSS, and JavaScript.
4. Compress responses: Use compression methods such as Gzip or Brotli where appropriate.
5. Optimize images: Reduce image file sizes and use efficient formats.

6. Use a Content Delivery Network (CDN): Deliver resources from servers closer to users.
7. Minimize CSS and JavaScript: Remove unnecessary code and reduce file sizes.
8. Use persistent connections: Reuse network connections where appropriate.
9. Use asynchronous loading: Load non-critical JavaScript without unnecessarily blocking page rendering.

Conclusion: Browser-server communication depends on DNS resolution, network connectivity, server processing, and browser rendering. Optimizing each stage helps reduce loading time and improves the user experience.